

Problem Description with Scenario

A retail store wants to maintain and manage the details of its products using an object-oriented approach. Each product has a **product name** and **price**.

Create a `Product` class with the following attributes:

- `name` – String
- `price` – Integer

Store the product records in an **array of `Product` objects** and perform the required operations on these records.

The following operations must be supported:

1. **ADD name price**
Add a new product with the given price.
If a product with the same name already exists, ignore the operation.
2. **UPDATE name price**
Update the price of an existing product.
If the product does not exist, ignore the operation.
3. **GET name**
Retrieve the price of the specified product.
If the product does not exist, store -1 as the result.

All product records must be maintained using an **array of `Product` objects**.

The program should process all operations and store the results of the `GET` operations in an integer array.

Input Format

- The first line contains an integer `N`, representing the number of operations.
- The next `N` lines contain one operation in one of the following formats:

```
ADD name price
UPDATE name price
GET name
```

Output Format

Print all the results generated by the `GET` operations in a single line.

The results should be displayed in array format:

```
[1200, 1500, -1]
```

If there are no GET operations, print:

```
[]
```

Constraints

- $1 \leq N \leq 1000$
- $0 \leq \text{price} \leq 1000000$
- $1 \leq \text{name.length}() \leq 20$
- The product name contains only alphabetic characters (A-Z, a-z).
- The solution must use an **array of Product objects**.
- List, HashMap, and other collection classes must not be used.

Sample Input

```
6
ADD Laptop 55000
ADD Mobile 25000
GET Laptop
UPDATE Laptop 60000
GET Laptop
GET Tablet
```

Sample Output

```
[55000, 60000, -1]
```

Explanation

The operations are processed as follows:

- ADD Laptop 55000 → Laptop is added with a price of 55000.
- ADD Mobile 25000 → Mobile is added with a price of 25000.
- GET Laptop → Laptop's price is 55000.
- UPDATE Laptop 60000 → Laptop's price is updated to 60000.
- GET Laptop → Laptop's updated price is 60000.
- GET Tablet → Tablet does not exist, so -1 is stored.

Therefore, the results of all GET operations are:

```
[55000, 60000, -1]
```